# Cryptography

3cut

June 2026

# Contents


**1 Introduction** **2**
- 1.1 About me . . . 2
- 1.2 Two Different Approach . . . 2
- 1.3 Quantum Computers . . . 2
- 1.4 Post-Quantum Cryptography . . . 3

**2 Current Cryptography** **4**
- 2.1 RSA . . . 4
  - 2.1.1 Key generation . . . 4
  - 2.1.2 Encryption and Decryption . . . 5
  - 2.1.3 Proof . . . 5
- 2.2 Discrete logarithm problem . . . 5
  - 2.2.1 Definition . . . 5
  - 2.2.2 Diffie-Hellman Key Exchange . . . 6
  - 2.2.3 Schnorr Signatures (Digital Signature Scheme) . . . 6
  - 2.2.4 ElGamal Encryption System . . . 8
  - 2.2.5 Elliptic Curves . . . 9

**3 Post-Quantum Cryptography** **12**
- 3.1 Post-Quantum Algorithm . . . 12
  - 3.1.1 Introduction . . . 12
  - 3.1.2 Shor's Algorithm . . . 12
  - 3.1.3 Grover's Algorithm . . . 12
- 3.2 NIST's standardized protocols . . . 12
  - 3.2.1 Introduction . . . 12
  - 3.2.2 ML-KEM: CRYSTALS-Kyber . . . 12
  - 3.2.3 KEM: HQC . . . 12
  - 3.2.4 ML-DSA: CRYSTALS-Dilithium . . . 12
  - 3.2.5 SLH-DSA: SPHINCS+ . . . 12

# 1. Introduction

## 1.1 About me

I'm a computer science student, currently studying at an engineering school. I love math, and during my studies, I discovered cryptography and the mathematical problems behind it. This sparked my curiosity and led me to post-quantum cryptography. Now, I'm trying to figure out what lies beneath these new protocols and truly understand the math problems that power them.

## 1.2 Two Different Approach

Before 1976, cryptography was entirely based on symmetric systems, where Alice and Bob shared a common secret: the private key. This single key was used for both encryption and decryption. However, the fundamental flaw was the key-distribution problem; they had to securely share this key beforehand, often requiring physical means like a "secure courier" or handwritten paper. In 1976, Whitfield Diffie and Martin Hellman introduced asymmetric cryptography. This revolutionary shift removed the need to share a secret key beforehand using two distinct but mathematically linked keys: a public key for encryption and a private key for decryption. This second approach still allows two people to securely compute a common secret key, such as for example the Diffie-Hellman Key Exchange.

In this document I will only talk about the Asymetric Protocols and the Digital Signature Schemes.

Digital Signature vouch for the author, it shows that the document has not been modified without him knowing.

## 1.3 Quantum Computers

Classical computers are based on the number of transistors inside them. Indeed, transistors let electrical current pass through them or not, representing a 0 or a 1—this is the bit. By miniaturizing them, more and more can be packed into computers, representing a vast number of states. Quantum computers, unlike classical ones, represent bits using other physical phenomena; these are known as qubits. These qubits have three distinct properties. Superposition allows one qubit to represent 0 and 1 at the same time, similar to the analogy of Schrödinger's cat, which was simultaneously dead and alive. Thus, $n$ qubits can represent $2^n$ states simultaneously. Interference

means that when a qubit is measured to determine its state, the outcome is a matter of probability of obtaining a 0 or a 1. Finally, through entanglement, a group of qubits can be linked together, allowing several operations to be performed on them at once.

## 1.4 Post-Quantum Cryptography

The cryptographic protocols widely used today, such as RSA and Elliptic Curve Cryptography (ECC), will not be resistant to cryptanalytic attacks by future quantum computers. Two primary quantum algorithms pose this threat: Shor's and Grover's algorithm. To get ahead of this vulnerability, the National Institute of Standards and Technology (NIST) initiated a global standardization project.

# 2. Current Cryptography

The two types of protocols that are being the most used are RSA and the protocols based on the Discrete Logarithm Problem.

## 2.1 RSA

Rivest Shamir Alderman protocol is based on the multiplication of two large prime numbers.

### 2.1.1 Key generation

1. **Choose** two distinct, large prime numbers $p, q \in \mathbb{P}$ such that:

   $$p \neq q$$

   These primes must be sufficiently large (typically 1024 bits or more each in modern RSA) to ensure that the modulus $n$ cannot be factored using classical factorization methods, such as **Pollard's rho algorithm**, which can efficiently find small prime factors.

2. **Calculate** the modulus $n$ and Euler's totient function $\phi(n)$:

   $$\begin{aligned} n &= p \cdot q \\ \phi(n) &= \phi(p) \cdot \phi(q) = (p-1)(q-1) \quad (\textit{Euler's totient indicator}) \end{aligned}$$

3. **Choose** an encryption exponent $e$ such that:

   $$1 < e < \phi(n) \quad \text{and} \quad \gcd(e, \phi(n)) = 1$$

4. **Choose** a decryption exponent $d$ such that:

   $$d \equiv e^{-1} \pmod{\phi(n)} \quad \text{and} \quad d < \phi(n)$$

   Thus, the cryptographic keys are defined as:

   $$\begin{aligned} \text{Public Key} &= (n, e) \\ \text{Private Key} &= (d, p, q) \end{aligned}$$

### 2.1.2 Encryption and Decryption

Given a message $m$ such that $m < n$.

#### Encryption

The ciphertext $c$ is computed using the public key $(n, e)$:

$$c \equiv m^e \pmod{n} \quad \text{with } c < n$$

#### Decryption

The original message $m$ is recovered using the private key $d$:

$$m \equiv c^d \pmod{n}$$

### 2.1.3 Proof

$$c^d \equiv (m^e)^d \equiv m^{ed} \pmod{n}$$

From the key generation phase, we know that $ed \equiv 1 \pmod{\phi(n)}$, which implies there exists an integer $k \in \mathbb{N}$ such that:

$$ed = 1 + k \cdot \phi(n) = 1 + k(p-1)(q-1)$$

So:

$$m^{ed} = m^{1+k(p-1)(q-1)} = m \cdot (m^{(p-1)(q-1)})^k$$

Assuming $m$ is not a multiple of $p$ ($m \not\equiv 0 \pmod{p}$) and $m$ is not a multiple of $q$ ($m \not\equiv 0 \pmod{q}$), we can apply **Fermat's Little Theorem**:

$$m^{p-1} \equiv 1 \pmod{p}$$
$$m^{q-1} \equiv 1 \pmod{q}$$

By applying these modular congruences simultaneously via **the Chinese Remainder Theorem**:

$$m^{(p-1)(q-1)} \equiv 1 \pmod{n}$$

Thus:

$$m^{ed} \equiv m \cdot (1)^k \equiv m \pmod{n}$$

Therefore $c^d \equiv m \pmod{n}$.

## 2.2 Discrete logarithm problem

### 2.2.1 Definition

Let $(G, *)$ a group. Let $b \in G$. For any positive integer $k$:

$$b^k = \underbrace{b \cdot b \cdot \cdots \cdot b}_{k \text{ factors}}$$

Let $b^{-k}$ denote the product of $b^{-1}$ with itself $k$ times. For $k = 0$: $b^0 = 1$.

Let $a \in G$. An integer $k$ that solves the equation $b^k = a$ is termed a **discrete logarithm** of $a$ to the base $b$. It can be written $k = \log_b a$.

### 2.2.2 Diffie-Hellman Key Exchange

The Diffie-Hellman protocol allows two parties to securely establish a shared secret over an insecure channel, which can be used as a key for symetric cryptography.

#### Setup

Alice and Bob agree on a public generator $g$ and a large prime number $p$, which define a cyclic group $G$.

#### Key Generation

1. Alice and Bob choose their respective private random numbers $a$ and $b \in G$:

$$\begin{aligned} \text{Alice's private key:} \quad & a \\ \text{Bob's private key:} \quad & b \end{aligned}$$

2. They compute and exchange their public keys:

$$\begin{aligned} \text{Alice's public key:} \quad & A = g^a \pmod p \\ \text{Bob's public key:} \quad & B = g^b \pmod p \end{aligned}$$

#### Shared Secret Computation

After exchanging their public keys, both parties can calculate the identical shared secret $K$:

$$\begin{aligned} K &= B^a \pmod p \\ &= A^b \pmod p \\ &= g^{ab} \pmod p \end{aligned}$$

### 2.2.3 Schnorr Signatures (Digital Signature Scheme)

The Schnorr signature scheme is a digital signature protocol known for its simplicity and mathematical efficiency. The protocol is structured as follows:

#### Public Parameters

Let $q$ be a large prime number ($q \in \mathbb{P}$). Let $G$ be a cyclic group generated by $g$, written as $G = \langle g \rangle$, where the order of the group is $q$ (i.e., $\#G = q$).

**Key Generation**

1. **Choose** a private signing key $x \in \mathbb{Z}_q^*$.
2. **Compute** the corresponding public verification key $y$:

$$y = g^{-x} \pmod{p}$$

**Signing**

To sign a message $M$, the signer performs the following steps:

1. Choose a random $k \in \mathbb{Z}_q^*$.
2. Compute the commitment value $r$:

$$r = g^k$$

3. Compute the challenge $e$ using a cryptographic hash function $H$, where $\|$ denotes concatenation:

$$e = H(r \parallel M)$$

4. Compute the proof value $s$:

$$s = k + xe \pmod{q}$$

The resulting signature pair sent with the message is $(s, e) \in \mathbb{Z}_q \times \mathbb{Z}_q$.

**Verification**

To verify the signature pair $(s, e)$ for a given message $M$, the verifier:

1. Reconstructs the commitment value $r_v$:

$$r_v = g^s \cdot y^e$$

2. Computes the verification challenge $e_v$:

$$e_v = H(r_v \parallel M)$$

If $e_v = e$, the signature is officially **verified**.

**Proof**

$$r_v = g^s \cdot y^e$$

And $s = k + xe$, $y = g^{-x}$, so:

$$r_v = g^{k+xe} \cdot (g^{-x})^e$$

Thus:

$$\begin{aligned} r_v &= g^k \cdot g^{xe} \cdot g^{-xe} \\ r_v &= g^k \cdot g^{xe-xe} \\ r_v &= g^k \cdot g^0 \\ r_v &= g^k \cdot 1 \\ r_v &= g^k \end{aligned}$$

Since the signer originally defined the commitment as $r = g^k$, we have shown that:

$$r_v = r$$

Therefore:

$$e_v = H(r_v \parallel M) = H(r \parallel M) = e$$

Thus, $e_v = e$.

### 2.2.4 ElGamal Encryption System

The ElGamal encryption scheme is an asymmetric key encryption algorithm based on the Diffie-Hellman key exchange. It operates over a cyclic group $G$ of prime order $q$ with generator $g$.

#### Key Generation

To generate a key pair, a user performs the following steps:

1. **Choose** a random private secret key $x$ from the set:

$$x \in \{1, \ldots, q-1\}$$

2. **Compute** the corresponding public verification key $h$:

$$h = g^x$$

$$\begin{aligned} \textbf{Public Key} &= (G, q, g, h) \\ \textbf{Private Key} &= x \end{aligned}$$

#### Encryption

To encrypt a message $M$:

1. **Choose** a random ephemeral key $y$ from the set:

$$y \in \{1, \ldots, q-1\}$$

2. **Compute** the shared secret $s$:
$$s = h^y$$

3. **Compute** the first component of the ciphertext $c_1$:
$$c_1 = g^y$$

4. **Compute** the second component of the ciphertext $c_2$:
$$c_2 = M \cdot s$$

$$\textbf{Ciphertext} = (c_1, c_2).$$

**Decryption**

To decrypt the ciphertext pair $(c_1, c_2)$, use the secret key $x$:

1. **Compute** the shared secret $s$ from the received $c_1$:
$$s = c_1^x$$

2. **Compute** the modular inverse of the shared secret: $s^{-1}$

3. **Recover** the original message $M$:
$$M = c_2 \cdot s^{-1}$$

**Proof**

$$s = c_1^x = (g^y)^x = g^{xy} \text{ and } h^y = (g^x)^y = g^{xy}$$

Therefore:
$$s = c_1^x = h^y.$$

Thus:
$$c_2 \cdot s^{-1} = (M \cdot s) \cdot s^{-1} = M \cdot (s \cdot s^{-1}) = M$$

### 2.2.5 Elliptic Curves

Elliptic Curve Cryptography is an approach to public-key cryptography based on the algebraic structure of elliptic curves over finite fields.

**Definition**

An elliptic curve is defined by a Weierstrass equation of the form:
$$y^2 = x^3 + ax + b$$

For cryptographic utility, the curve parameters must satisfy $4a^3 + 27b^2 \neq 0$ to ensure the curve is smooth (has no cusps or self-intersections).

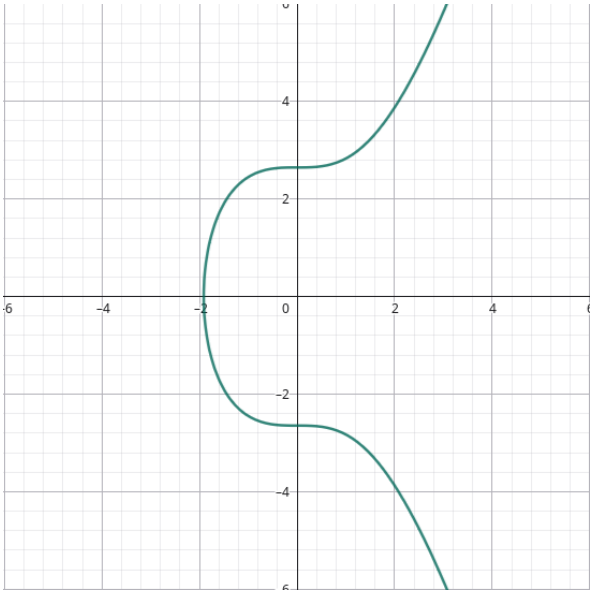



Figure: Equation used in the Digital Signature Scheme of Bitcoin's Blockchain ($y^2 = x^3 + 7$)

### Application in Cryptography

The security of Elliptic Curve Cryptography relies on the Elliptic Curve Discrete Logarithm Problem.

### Key Generation

1. Given a globally known, public base point $P$ on an elliptic curve $E$ over a finite field.
2. A user selects a private **secret key** $k$, which is a randomly chosen integer:

$$k \in [1, \#E - 1]$$

3. The corresponding **public key** $Q$ is computed via scalar point multiplication:

$$Q = k \cdot P$$

$$\textbf{Public Key} = Q$$
$$\textbf{Private Key} = k$$

### Encryption

To encrypt a message, it must first be mapped to a point $M$ on the elliptic curve $E$. To encrypt this message point $M$ for the recipient, the sender performs the following steps:

1. **Choose** a random ephemeral integer $r$ such that:

$$r \in [1, \#E - 1]$$

2. **Compute** the first ciphertext point $C_1$, which encodes the ephemeral key:

$$C_1 = r \cdot P$$

3. **Compute** the second ciphertext point $C_2$, which masks the message point using the recipient's public key $Q$:

$$C_2 = M + (r \cdot Q)$$

$$\textbf{Ciphertext} = (C_1, C_2)$$

**Decryption**

To decrypt the ciphertext pair $(C_1, C_2)$ and recover the original message point $M$:

1. **Compute** the shared secret point by multiplying the received $C_1$ with the secret key $k$:

$$\text{Shared Secret} = k \cdot C_1$$

2. **Subtract** this shared secret point from $C_2$ to isolate and recover $M$:

$$M = C_2 - (k \cdot C_1)$$

**Proof**

$$C_2 - (k \cdot C_1) = \big(M + (r \cdot Q)\big) - (k \cdot C_1)$$

And $Q = k \cdot P$, $C_1 = r \cdot P$, so:

$$M + \big(r \cdot (k \cdot P)\big) - \big(k \cdot (r \cdot P)\big)$$

$$M + (rk \cdot P) - (rk \cdot P) = M$$

# 3. Post-Quantum Cryptography

## 3.1 Post-Quantum Algorithm

### 3.1.1 Introduction

The current best known factoring algorithm that could potentially break RSA cryptosystem is called General Number Field Sieve.

### 3.1.2 Shor's Algorithm

### 3.1.3 Grover's Algorithm

## 3.2 NIST's standardized protocols

### 3.2.1 Introduction

### 3.2.2 ML-KEM: CRYSTALS-Kyber

### 3.2.3 KEM: HQC

Coming soon...

### 3.2.4 ML-DSA: CRYSTALS-Dilithium

Coming soon...

### 3.2.5 SLH-DSA: SPHINCS+

Coming soon...

# Bibliography


[1] Digital signature — Wikipedia.

[2] Key encapsulation mechanism — Wikipedia.

[3] Public-key cryptography — Wikipedia.

[4] RSA (cryptosystem) — Wikipedia.

[5] Discrete logarithm — Wikipedia.

[6] Diffie–Hellman key exchange — Wikipedia.

[7] Schnorr signature — Wikipedia.

[8] ElGamal encryption — Wikipedia.

[9] Elliptic-curve cryptography — Wikipedia.

[10] "Viens, on recode Bitcoin pour le comprendre" — V2F, Youtube.